Building Mapped Types and Utility Types

In addition to limiting variables to a set of specific literal values and defining enums, TypeScript allows you to define types that are based on other types, kind of like super-powered generics. These are called *mapped types*. TypeScript also has a bunch of predefined mapped types that it calls *utility types*.

So let's say we have our existing type TicketData:

```
interface TicketData {
  id: number
  row: number
  number: number
  status: TicketStatus
}
```

And let's say we have a data source that is unreliable and we want to make all the fields optional for data coming from that source. We could create a new type:

```
interface OptionalTicketData {
  id?: number
  row?: number
  number?: number
  status?: TicketStatus
}
```

But that's kind of verbose, and if the TicketData type changes, we also have to change the new type.

TypeScript lets us do this more generically:

```
type OptionalTicketData = [P in keyof TicketData]?: TicketData[P]
```

I don't want to get wildly hung up on the syntax here, but basically we have a variable P that iterates over each type (in keyof and then adds the optional signifier to each type ?). Essentially what we're getting here is a new type where every key in the old type is optional. This is something kind of like a type declaration and kind of like a function declaration.